

Lab 1 — PyTorch: Tensors, Autograd and a First MLP

Machine Learning II · Tutorial

Rodrigo Grijalba (teaching assistant) · Alexander Quispe (instructor)

Wednesday, October 7, 2026 · 19:00–22:00

Goal of the session. Move from numpy to PyTorch without losing sight of the mathematics: every gradient PyTorch computes today, you will first compute by hand. By the end you will have trained a single neuron and a two-layer network with a training loop you wrote yourself, and checked its gradients against `autograd`.

Materials

- `homework_1_code.ipynb` — the instructor’s Homework 1 notebook (deep-learning-up). Today: Exercises 1–3.
- `homework_1_theory.pdf` — the theory problems; Problem 1 (manual backpropagation) is worked in class.
- `data/synthetic_2d.csv`, `data/diabetes.csv`.
- Lecture 1 of ML II (perceptron, XOR, chain rule, backpropagation).

1 Environment (15 min)

Colab: Runtime → Change runtime type → GPU (T4). Local: `pip install torch torchvision`. Verify:

```
import torch; print(torch.__version__, torch.cuda.is_available())
```

Nothing today needs a GPU; the point is to know how to move a tensor to one (`.to(device)`) before Lab 2 does.

2 Tensors (30 min)

A tensor is a numpy array that (i) can live on a GPU and (ii) can record the operations applied to it.

Task 1 — the same computation, twice

Redo Lab 1 of ML I in torch: the 5×2 design matrix, $\mathbf{X}^\top \mathbf{X}$, and the normal-equation solution with `torch.linalg.solve`. Confirm (1.49, 0.75). Note the three things that differ from numpy: `dtype` defaults to `float32`, `@` works the same, and `.numpy()` converts back.

Task 2 — broadcasting and shapes

Build a batch of $B = 64$ inputs $\mathbf{X} \in \mathbb{R}^{64 \times p}$, a weight matrix $\mathbf{W} \in \mathbb{R}^{p \times h}$ and a bias $\mathbf{b} \in \mathbb{R}^h$. Compute $\mathbf{XW} + \mathbf{b}$ and explain which axis the bias broadcasts along. Then transpose one operand deliberately and read the error message — you will see it again.

3 Autograd (45 min)

Task 3 — the derivative you know

Let $f(\mathbf{w}) = \frac{1}{2} \|\mathbf{y} - \mathbf{Xw}\|^2$. Create `w = torch.zeros(p, requires_grad=True)`, compute `f`, call `f.backward()`, and compare `w.grad` with $-\mathbf{X}^\top(\mathbf{y} - \mathbf{Xw})$ from ML I, Lecture 1. They must agree to `float32` precision.

Task 4 — a single neuron (Exercise 2)

Following Exercise 2 of the notebook: sigmoid neuron, BCE loss, gradient descent on `synthetic_2d.csv`. Before running `.backward()`, write the gradient by hand using the identity from ML I, Lecture 2: $\partial \ell / \partial \mathbf{w} = (\sigma(\mathbf{w}^\top \mathbf{x}) - y) \mathbf{x}$. Assert the two agree.

Task 5 — gradient descent, from ML I to torch (Exercise 1)

Exercise 1: OLS vs. gradient descent on `diabetes.csv`. Standardize, run GD with three learning rates, and reproduce the divergence above $2/\lambda_{\max}$ you saw in ML I — now with `torch.optim.SGD` doing the update. Read the source of `SGD.step` once; it is twelve lines.

4 Manual Backpropagation (50 min)

Task 6 — Exercise 3, and Problem 1 of the theory sheet

A two-layer network with sigmoid hidden units and a sigmoid output, BCE loss. Work Problem 1(a)–(c) on paper first (the “beautiful cancellation” $\partial \ell / \partial z = \hat{y} - y$ for sigmoid + BCE, then the two weight gradients by the chain rule). Then implement the forward and backward passes in numpy without autograd, train, plot the decision boundary. Finally, rebuild the same network with `torch.nn.Module` and check that `autograd`’s gradients match yours at the initial weights to 10^{-6} .

The most common bug: forgetting `optimizer.zero_grad()` before `.backward()`. Gradients accumulate in PyTorch. Try it once deliberately and watch the loss curve.

5 Exercises (to submit before Lab 2)

Exercise 1. Theory sheet, Problems 2 (activation derivatives, vanishing gradient) and 3 (OR with ReLU).

Exercise 2. Notebook Exercise 4.A: an MLP with at most 100 hidden units on MNIST. Report test accuracy and the number of parameters. (The MNIST download is automatic through `torchvision`.)

Submission

Notebook `lab01_lastname_firstname.ipynb` and a PDF of the paper work, before Wednesday, October 14, 18:00.